

Lab 3

Семсічко Лідія

Запити та результати

GET http://localhost:8000/health

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8000/health
- Params:** The 'Params' tab is active, showing a table with headers 'Key' and 'Value'.
- Body:** The 'Body' tab is active, showing a JSON response:

```
{  "status": "ok",  "service": "facade-service"}
```

POST http://localhost:8000/transaction

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8000/transaction
- Body Type:** raw (selected)
- Request Body:**

```
{  "user_Id": "user1",  "amount": 10,  "transaction_id": "tx1"}
```
- Response:** The 'Body' tab is active, showing a JSON response:

```
{  "transaction_id": "tx1",  "logging": {    "status": "stored",    "service": "logging-service-3",    "transaction_id": "tx1"  },  "balance": 10010.0}
```

GET http://localhost:8000/accounts

HTTP **http://localhost:8000/accounts**

GET **http://localhost:8000/accounts**

Docs Params Authorization Headers (8) Body • Scripts

Query Params

Key
Key

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "balances": {
3     "user1": 10.0
4   }
5 }
```

Повторно (бачимо ідемпотентність)

POST http://localhost:8000/s • POST http://localhost:8000/t • +

HTTP **http://localhost:8000/transaction**

POST **http://localhost:8000/transaction**

Docs Params Authorization Headers (8) Body • Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "transaction_id": "tx1",
3   "logging": {
4     "status": "duplicate",
5     "service": "logging-service-1",
6     "transaction_id": "tx1"
7   },
8   "balance": 10.0
9 }
```

Від'ємна транзакція

POST <http://localhost:8000/transaction>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
5 }
```

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "transaction_id": "tx2",
3   "logging": {
4     "status": "stored",
5     "service": "logging-service-3",
6     "transaction_id": "tx2"
7   },
8   "balance": 5.0
9 }
```

HTTP <http://localhost:8000/accounts>

GET <http://localhost:8000/accounts>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
5 }
```

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "balances": {
3     "user1": 5.0
4   }
5 }
```

Невідомий юзер

HTTP <http://localhost:8000/user/unknown>

GET <http://localhost:8000/user/unknown>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL [JSON](#) ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
}
```

Body Cookies Headers (4) Test Results ↻

[{} JSON](#) ▾ ▶ Preview Visualize ▾

```
1 {
2   "balance": 0.0,
3   "transactions": []
4 }
```

Метрики (це вже після великої кількості експериментів та лоад тестів)

HTTP <http://localhost:8000/metrics>

GET <http://localhost:8000/metrics>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL [JSON](#) ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
}
```

Body Cookies Headers (4) Test Results ↻

[{} JSON](#) ▾ ▶ Preview Visualize ▾

```
1 {
2   "logging": {
3     "calls": 100005,
4     "total_s": 7013.152055817286,
5     "avg_ms": 70.12801415746499
6   },
7   "counter": {
8     "calls": 100106,
9     "total_s": 8716.899669386998,
10    "avg_ms": 87.07669539674941
11  }
12 }
```

Pecet

HTTP **http://localhost:8000/system/reset**

POST **http://localhost:8000/system/reset**

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
```

Body Cookies Headers (4) Test Results ↻

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "status": "reset_all"
3 }
```

HTTP **http://localhost:8000/accounts**

GET **http://localhost:8000/accounts**

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "user_Id": "user1",
3   "amount": -5,
4   "transaction_id": "tx2"
```

Body Cookies Headers (4) Test Results ↻

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "balances": {}
3 }
```

Відмовостійкість

```
PS C:\Users\user\Desktop\apz\micro-service> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
0f49eb140291   micro-service-facade-service        "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  0.0.0.0:8000->8000/tcp             facade-service
9812a7003464   micro-service-logging-service-1     "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  8000/tcp                           logging-service-1
91686ced657b   micro-service-counter-service       "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  8000/tcp                           counter-service
8a7bdcd6f72    micro-service-logging-service-2     "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  8000/tcp                           logging-service-2
bc0e9ab36063   micro-service-logging-service-3     "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  8000/tcp                           logging-service-3
e7fb850dba74   postgres:15                         "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:5432->5432/tcp             postgres
53f783a1e353   hazelcast/hazelcast:5.3.6          "hz start"              2 minutes ago  Up 2 minutes  0.0.0.0:5702->5701/tcp             hazelcast-2
600d7b9970bb   hazelcast/hazelcast:5.3.6          "hz start"              2 minutes ago  Up 2 minutes  0.0.0.0:5703->5701/tcp             hazelcast-3
821903d78a76   hazelcast/hazelcast:5.3.6          "hz start"              2 minutes ago  Up 2 minutes  0.0.0.0:5701->5701/tcp             hazelcast-1
PS C:\Users\user\Desktop\apz\micro-service>
```

```
PS C:\Users\user\Desktop\apz\micro-service> docker stop logging-service-1
logging-service-1
```

HTTP <http://localhost:8000/transaction>

POST <http://localhost:8000/transaction>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "user_Id": "fail_test",
3   "amount": 10,
4   "transaction_id": "fail1"
}
```

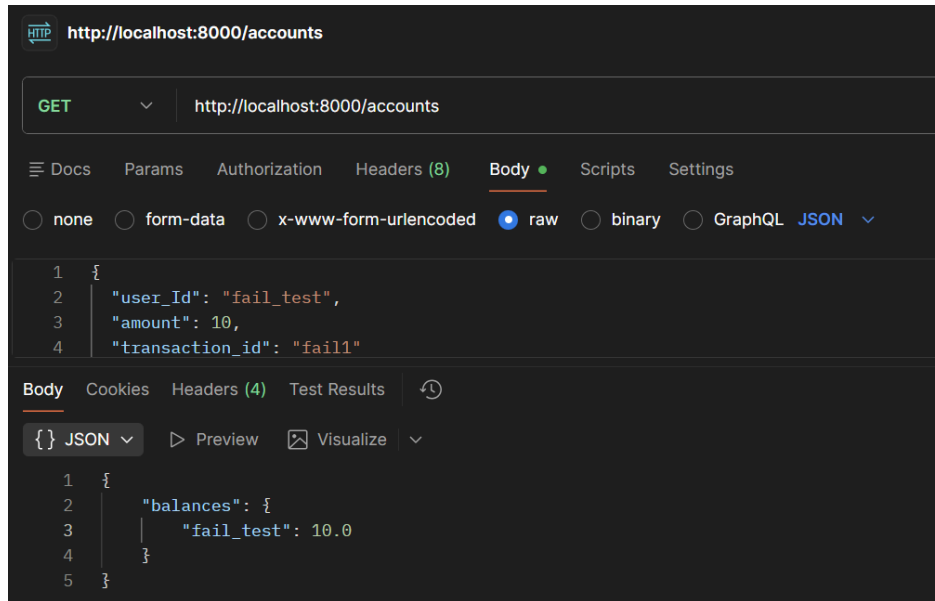
Body Cookies Headers (4) Test Results ↺

{} **JSON** ▾ ▶ Preview 🖼 Visualize ▾

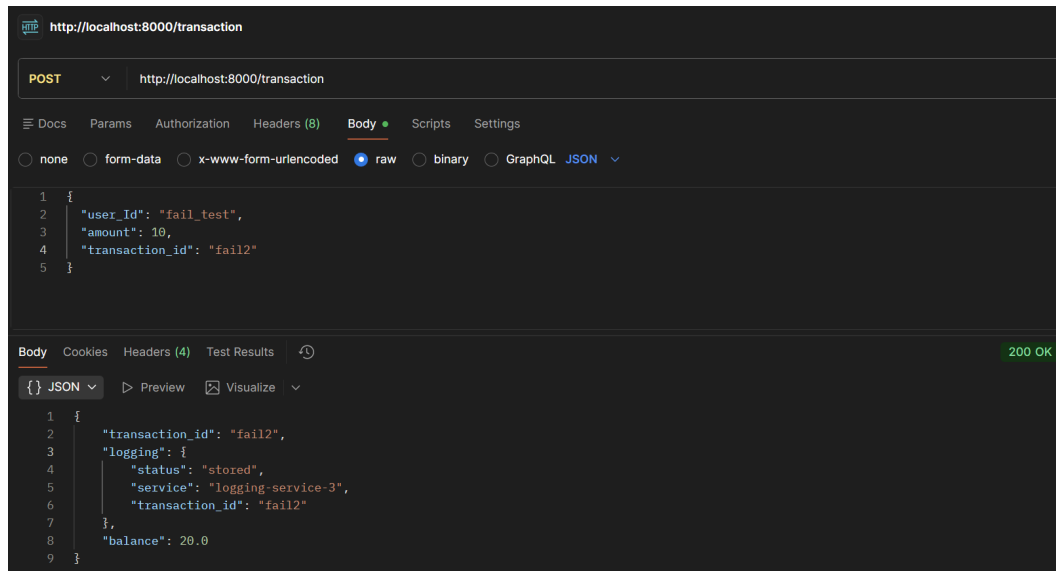
```
1 {
2   "transaction_id": "fail1",
3   "logging": {
4     "status": "stored",
5     "service": "logging-service-3",
6     "transaction_id": "fail1"
7   },
8   "balance": 10.0
9 }
```

Було вимкнено один екземпляр logging-service.
Система продовжила обробляти запити, використовуючи інші доступні екземпляри.

Втрата одного сервісу не вплинула на коректність роботи системи.

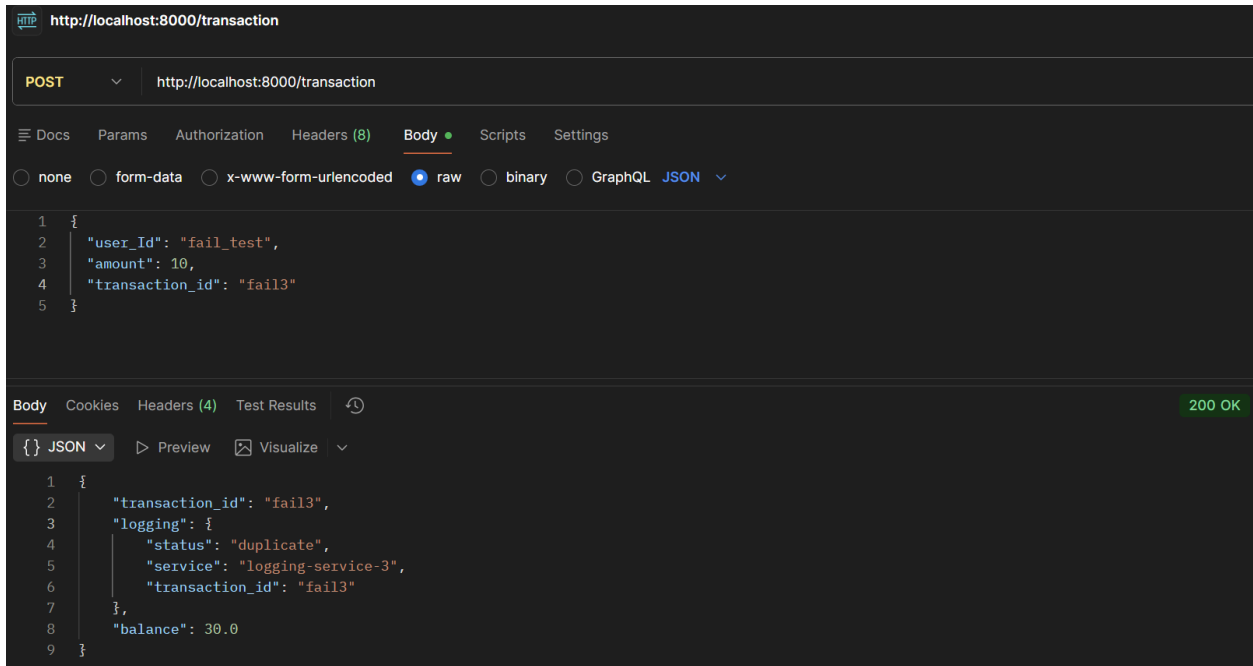


```
PS C:\Users\user\Desktop\apz\micro-service> docker stop logging-service-1
logging-service-1
PS C:\Users\user\Desktop\apz\micro-service> docker stop logging-service-2
logging-service-2
```



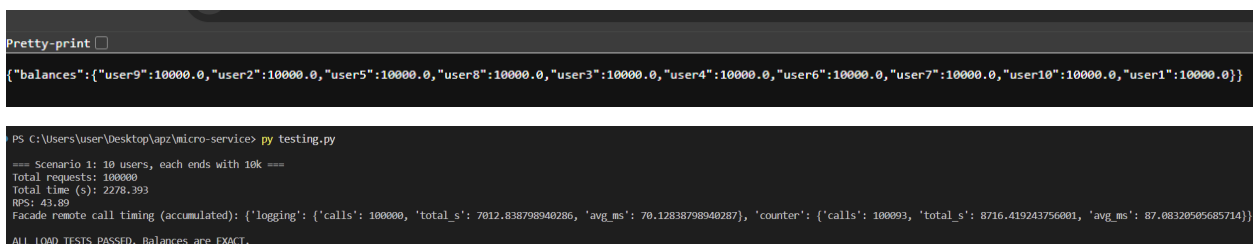
З двома вимкненими теж працює

```
PS C:\Users\user\Desktop\apz\micro-service> docker stop hazelcast-1
hazelcast-1
```



Було вимкнено один/два вузли Hazelcast.
Система продовжила роботу, оскільки кластер автоматично перерозподіляє дані між доступними нодами.
Логи не були втрачені, транзакції оброблялись коректно.

Лoad тестування



У порівнянні з попередньою лабораторною роботою спостерігається зниження продуктивності системи.

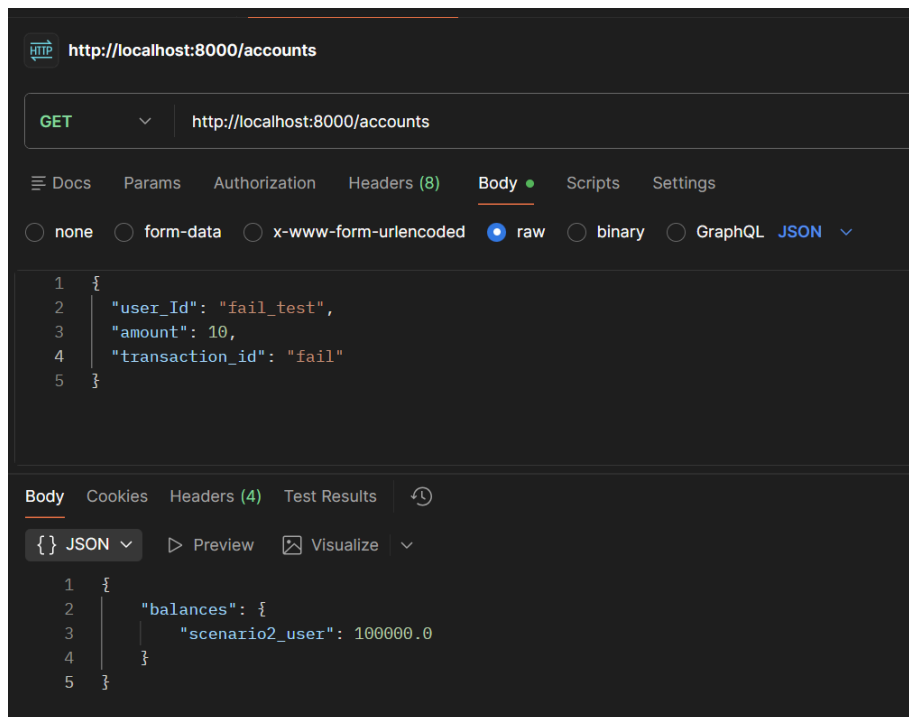
Пропускна здатність зменшилась приблизно з 76 до 44 запитів на секунду, а загальний час виконання зріс майже вдвічі.

Основною причиною є ускладнення архітектури: додавання кількох logging-service, використання Hazelcast та збільшення кількості мережевих взаємодій між сервісами.

Середній час викликів також значно зріс (з ~15 мс до 70–87 мс), що свідчить про додаткові накладні витрати на міжсервісну комунікацію.

Незважаючи на зниження продуктивності, система забезпечує коректність обробки всіх транзакцій, що є більш критичним для реальних distributed систем.

Сценарій 2



```
PS C:\Users\User\Desktop\apz\micro-service> py testing.py
=== Scenario 2: 1 user, ends with 100k ===
Total requests: 100000
Total time (s): 2209.269
RPS: 45.26
Facade remote call timing (accumulated): {'logging': {'calls': 100001, 'total_s': 6602.176477814452, 'avg_ms': 66.02110456709885}, 'counter': {'calls': 100005, 'total_s': 8838.778366554323, 'avg_ms': 88.33036892574151}}
ALL LOAD TESTS PASSED. Balances are EXACT.
```

У другому сценарії після ускладнення архітектури спостерігається подальше зниження продуктивності системи. Пропускна здатність зменшилась з приблизно 54 до 45 запитів на секунду, а загальний час виконання зріс з 1834 до 2209 секунд.

Середній час викликів значно збільшився (з ~18–19 мс до 66–88 мс), що пов'язано з додатковими мережевими викликами між сервісами та використанням розподілених компонентів (Hazelcast, кілька logging-service).

Також на продуктивність впливає механізм блокування в counter-service, який забезпечує відсутність race condition, але серіалізує доступ до одного ресурсу. Незважаючи на це, система гарантує повну коректність результатів навіть при високому рівні конкурентності.

Висновок

Попри зростання затримок і зниження пропускнуої здатності, система забезпечує коректність обробки транзакцій, що є ключовим пріоритетом для надійних distributed систем.